

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к индивидуальному домашнему заданию
по дисциплине
«Введение в нереляционные системы управления базами данных»
Тема: БД для наблюдений за животными

Студент гр. 3341		Трофимов В.О.
Студент гр. 3341		Кудин А.А.
Студент гр. 3341		Мальцев К.Л.
Преподаватель		Заславский М.М.

ЗАДАНИЕ

Тема проекта: БД для наблюдений за животными

Исходные данные:

Задача - создать веб-приложение, в котором можно делиться своими наблюдениями за животными (фото, видео, отметки на карте, лайки, комментарии).

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания: 16.02.26

Дата сдачи реферата: 20.05.26

Дата защиты реферата: 20.05.26

Студент гр. 3341		Трофимов В.О.
Студент гр. 3341		Кудин А.А.
Студент гр. 3341		Мальцев К.Л.
Преподаватель		Заславский М.М.

АННОТАЦИЯ

В рамках курса разработано веб-приложение командой по теме «Приложение наблюдений» с использованием нереляционной СУБД. Приложение реализует интерфейсы и сценарии, описанные в макете и сценарии использования: страницы входа и регистрации, лента постов с фильтрами и возможностью массового импорта/экспорта, пошаговый процесс создания наблюдения (выбор типа, заголовок/описание, загрузка фото, место, теги), профиль пользователя с редактированием и статистикой, детальный просмотр поста с лайками и комментариями. Предусмотрены CRUD-операции для постов и комментариев, поиск и фильтрация ленты по типу, тегам, автору и диапазону дат, а также экспорт/импорт данных в формате JSON для администраторов.

<https://github.com/moevm/nsql1h26-anim>

ANNOTATION

As part of the course, a team developed a web application on the topic "Observation App" using a NoSQL database. The application implements the interfaces and use cases described in the UI mockup and scenarios: login and registration pages, a posts feed with filters and bulk import/export capabilities, a step-by-step observation creation flow (select type, title/description, upload photo, location, tags), a user profile with editing and statistics, and a detailed post view with likes and comments. CRUD operations for posts and comments are provided, along with search and filtering of the feed by type, tags, author, and date range, and JSON import/export functionality for administrators.

<https://github.com/moevm/nsql1h26-anim>

СОДЕРЖАНИЕ

Введение	6
1. Постановка задачи	8
1.1. Предлагаемое решение	8
1.2. Качественные требования к решению	8
2. Сценарии использования	10
2.1. Экраны	10
2.2. Сценарии	11
3. Модель данных	13
4. Разработанное приложение	15
4.1. Краткое описание	15
4.2. Используемые технологии	15
4.3. Снимки экрана приложения	15
5. Выводы	16
Литература	17
Приложение А. Документация по сборке и развертыванию	18
Приложение Б. Инструкция для пользователя	19

Введение

Данные наблюдений естественно принимают графовую форму: наблюдения, пользователи, места и теги и их взаимосвязи (кто сделал наблюдение, где, с какими тегами, комментарии и лайки) моделируются как узлы (посты, пользователи, места, теги) и ребра (`authored`, `located_at`, `tagged_with`, `commented_on`, `liked`). Реляционные СУБД требуют множества таблиц и сложных JOIN-операций для представления и обхода таких связей, что снижает производительность при глубоком или широком обходе и усложняет формулировку запросов.

Графовые базы данных хранят связи как объекты первого класса, что позволяет выполнять переходы по графу с постоянной стоимостью на шаг независимо от общего объёма данных. Neo4j — одна из зрелых и широко используемых графовых СУБД — предоставляет декларативный язык Cypher для описания паттернов в графе; запросы на Cypher для поиска связанных наблюдений, цепочек взаимодействий или наиболее влиятельных пользователей обычно короче и читабельнее эквивалентов на SQL с рекурсивными CTE.

Приложение «Animal» содержит тысячи постов, пользователей, тегов и мест, а также множество перекрёстных связей (комментарии, лайки, повторное использование тегов и мест), что делает его практическим примером для демонстрации преимуществ графовых СУБД при работе с сетевыми и иерархическими данными.

Цель работы — разработка веб-приложения для хранения, визуализации и анализа данных наблюдений с использованием Neo4j. Приложение обеспечивает полный цикл работы с данными: импорт/экспорт в

машиночитаемых форматах (JSON), ручное добавление и редактирование постов и связанных сущностей, поиск и фильтрацию по типу, тегам, автору и диапазону дат, интерактивную визуализацию связей в виде графа и формирование статистики использования.

Выбранный стек технологий: Neo4j Community Edition в качестве СУБД, Python с FastAPI для REST API, React для фронтенда. Все компоненты контейнеризированы через Docker и оркестрированы с помощью Docker Compose для упрощённого развёртывания и воспроизводимости среды.

В пояснительной записке описаны постановка задачи, макет и сценарии использования, модель данных (графовая vs реляционная) с их сравнительным анализом, архитектура и применённые технологии, а также выводы по результатам реализации.

1. Постановка задачи

Необходимо разработать веб-приложение для хранения, визуализации и анализа данных наблюдений с использованием графовой СУБД Neo4j.

1.1. Предлагаемое решение

Предлагаемое решение — трёхзвенное веб-приложение, состоящее из:

Neo4j 5.18 Community Edition в качестве графовой БД для хранения сущностей (посты, пользователи, места, теги, комментарии, лайки) и связей между ними в виде узлов и ребер.

Серверная часть (backend) на Python 3.12 с FastAPI, предоставляющая REST API для всех операций над данными (CRUD для постов, пользователей, тегов, мест; операции импорта/экспорта; поиск и фильтрация).

Клиентская часть (frontend) — одностраничное приложение (SPA) на React 19 с Vite 8, реализующее интерфейсы и сценарии использования: страницы входа/регистрации, лента постов с фильтрами, пошаговое добавление наблюдения, профиль пользователя, детальный просмотр поста и интерактивная визуализация связей.

Все компоненты контейнеризированы с Docker и оркестрируются через Docker Compose — одноразовый запуск для развертывания среды.

1.2. Качественные требования к решению

Удобство использования: интуитивный интерфейс с навигацией (Лента / Профиль), интерактивная визуализация связей (масштабирование, перетаскивание), модальные окна для быстрого просмотра и редактирования информации без перехода между страницами.

Контейнеризация: запуск всех компонентов (Neo4j, backend, frontend) одной командой через Docker Compose без ручной установки зависимостей.

Импорт/экспорт данных: поддержка форматов JSON для совместимости с внешними системами и возможностью массового импорта/экспорта ленты (для админов).

Целостность данных: валидация на клиенте и сервере (форматы полей, обязательные атрибуты), каскадное удаление связанных сущностей при удалении поста/пользователя, предзаполнение базы тестовыми данными для демонстрации.

Функциональные требования (кратко): CRUD для постов и комментариев, поиск и фильтрация ленты по типу, тегам, автору и диапазону дат, управление профилем пользователя, лайки/комментарии, статистика и экспорт/импорт.

Производительность и масштабируемость: оптимизация запросов Cypher для типичных сценариев (фильтрация ленты, выборка постов пользователя, подсчёт статистики), использование индексов в Neo4j для ускорения поиска.

Безопасность и отказоустойчивость: базовая аутентификация/авторизация для операций редактирования и админских функций, корректная обработка ошибок и возврат валидных HTTP-статусов.

2. Сценарии использования

2.1. Экраны

Страница логина: поле Email, поле Пароль (по умолчанию скрытое) с иконкой глаза для переключения видимости, кнопка «Войти» и ссылка «Нет аккаунта? Зарегистрироваться».

Страница логина — просмотр пароля (состояние): при активной иконке глаза пароль отображается текстом; повторное нажатие скрывает его.

Страница регистрации: поле Email, поле Пароль (скрытое), кнопка «Зарегистрироваться», ссылка «Уже есть аккаунт? Войти».

Лента постов (главный экран после входа): хедер с вкладками «Лента» и «Профиль», кнопки Экспорт/Импорт ленты в формате .json для админов, блок профиля с кнопкой «Выйти», кнопка «Добавить», кнопка «Фильтры» для открытия панели фильтрации и список карточек наблюдений.

Фильтры: выбор типа животного, сортировка по тегу/автору/дате.

Создание наблюдения – шаг 1: выбор типа животного, поле Заголовок, поле Описание и кнопка «Далее» для перехода к шагу 2.

Создание наблюдения – шаг 2: загрузка фото, поле Место наблюдения, поле Теги с кнопкой добавления, кнопка «Опубликовать» для публикации и возврата в ленту, кнопка «Назад» для возврата к шагу 1.

Профиль пользователя: отображаются имя, «о себе» и местоположение; кнопка «Изменить профиль» делает поля редактируемыми и позволяет сменить аватар; список опубликованных пользователем наблюдений; блок «Статистика» с счётчиками наблюдений, лайков и комментариев.

Страница поста: отображаются тип животного, заголовок, описание, фото, место, теги и автор; кнопка «Назад» возвращает на предыдущий экран; кнопка «Лайк» увеличивает счётчик; секция комментариев содержит список комментариев, поле ввода и кнопку отправки.

2.2. Сценарии

UseCase-01 — Вход в аккаунт

Откройте приложение: появляется страница логина; введите email и затем пароль (по умолчанию символы скрыты); при желании нажмите иконку глаза, чтобы отобразить пароль текстом, и нажмите её снова, чтобы скрыть; после ввода данных нажмите «Войти» — система валидирует данные и при успехе переходит в ленту.

UseCase-02 — Переход к регистрации

На странице логина нажмите «Нет аккаунта? Зарегистрироваться», после чего система откроет страницу регистрации.

UseCase-03 — Регистрация нового пользователя

На странице регистрации введите email и пароль (символы скрыты), затем нажмите «Зарегистрироваться» — при успешном создании аккаунта система автоматически откроет ленту.

UseCase-04 — Возврат к логину со страницы регистрации

На странице регистрации нажмите «Уже есть аккаунт? Войти», после чего откроется страница логина.

UseCase-05 — Фильтрация ленты

На ленте нажмите «Фильтры», в панели фильтров выберите тип животного, укажите сортировку и диапазон дат и при необходимости введите название места, затем нажмите «Применить» — система вернёт вас в ленту, где будут показаны отфильтрованные и отсортированные записи.

UseCase-06 — Создание наблюдения

В ленте нажмите «Добавить», на шаге 1 выберите тип животного и заполните заголовок и описание, затем нажмите «Далее»; на шаге 2 загрузите фото, заполните поле места наблюдения и добавьте теги, после чего нажмите «Опубликовать» — наблюдение будет опубликовано, и вы вернётесь в ленту.

UseCase-07 — Возврат на шаг 1 при создании

Если вы на шаге 2 создания и нажмёте «Назад», система вернёт вас на шаг 1, сохранив введённые поля (тип, заголовок, описание).

UseCase-08 — Переход в профиль

В ленте нажмите вкладку «Профиль» в хедере, и откроется страница профиля текущего пользователя.

UseCase-09 — Редактирование профиля

На странице профиля нажмите «Изменить профиль», отредактируйте поля «о себе» и «местоположение» (и при необходимости смените аватар), затем сохраните изменения — профиль обновится и отобразит новые данные.

UseCase-10 — Просмотр поста из ленты

В ленте нажмите карточку наблюдения — откроется страница поста с полной информацией.

UseCase-11 — Просмотр поста из профиля

В профиле нажмите на одно из своих наблюдений — откроется соответствующая страница поста.

UseCase-12 — Лайк наблюдения

На странице поста нажмите «Лайк» — лайк будет засчитан, и счётчик обновится.

UseCase-13 — Оставить комментарий

На странице поста введите текст в поле комментария и нажмите «Отправить» — комментарий добавится в список.

UseCase-14 — Удаление своего поста

Если вы просматриваете свой пост, нажмите «Удалить пост» — система удалит пост и вернёт вас в ленту.

UseCase-15 — Удаление комментария на своём посте

Если вы на своём посте у нужного комментария нажмите «Удалить», комментарий будет удалён из списка.

3. Модель данных

На изображениях ниже показаны модели данных из draw.io:

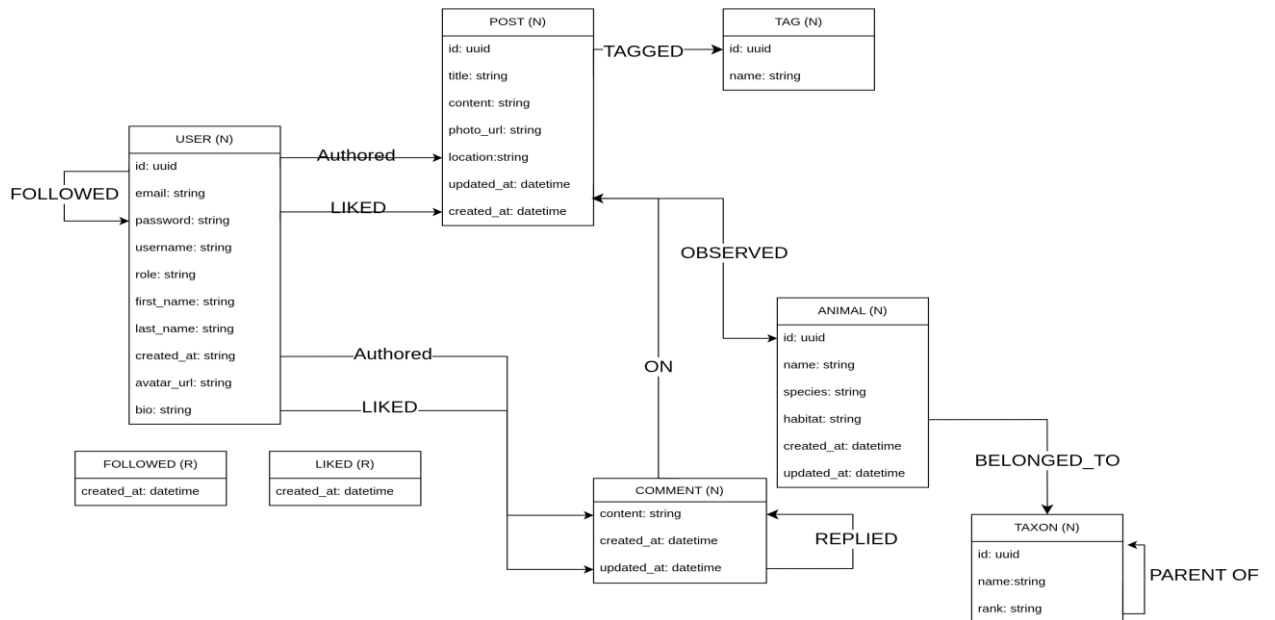


Рис. 1. Модель Neo4j

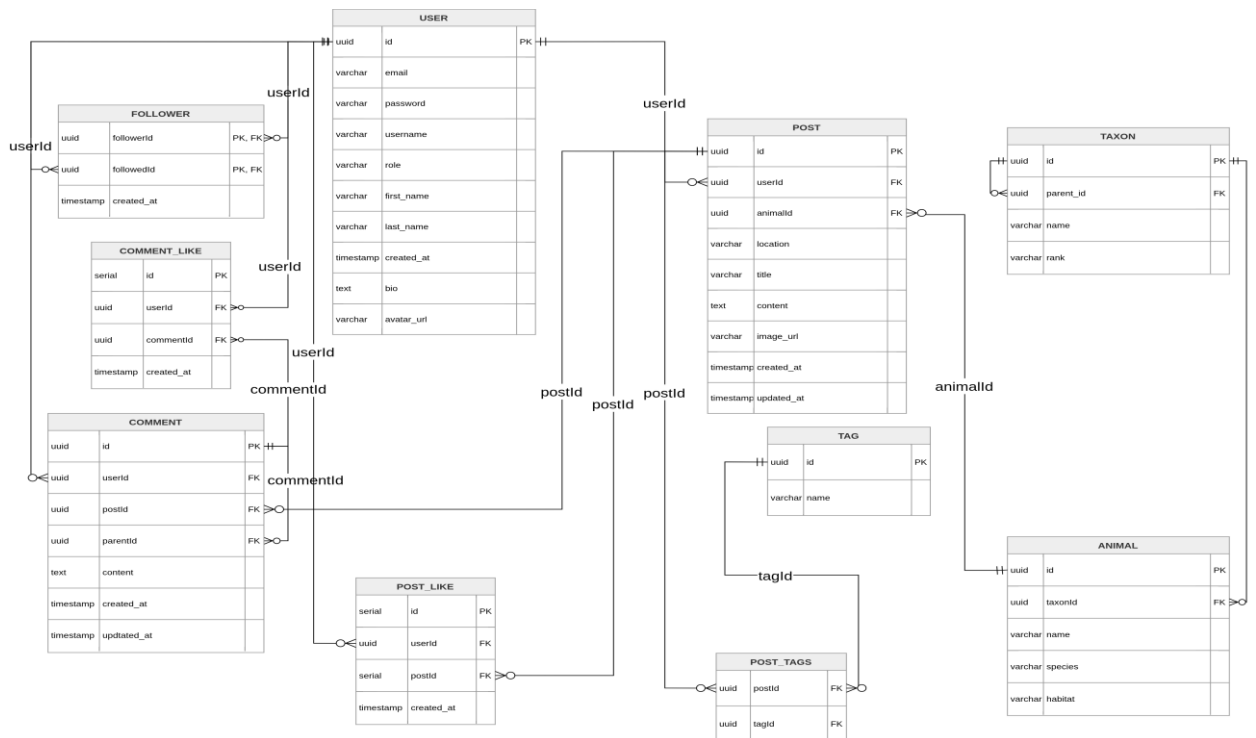


Рис. 1. Модель PostgreSQL

На таблице ниже показано сравнение моделей в ms для выборки запросов из представленных use-case-ов:

Сравнение моделей		
Use Case / Query Description	Postgres (ms)	Neo4j (ms)
UC-01: Login (Find User by Email)	1.907	8.160
UC-05: Filter Feed	1.605	15.514
UC-06: Social Recommendations	0.975	6.576
UC-07: Deep Taxon Tree Search (Recursive)	2.144	5.921
UC-10: Post Detail	1.967	7.188
UC-14: Social Recommendations (FOF)	1.342	6.230
UC-15: Taxonomy Search (Recursive Tree)	1.988	6.137

4. Разработанное приложение

4.1. Краткое описание

Приложение реализовано в виде трехконтейнерной архитектуры, управляемой через Docker Compose:

1) db – контейнер с графовой СУБД Neo4j 5.18 Community Edition. Хранит все данные о персонах и связях между ними. Данные персистируются через Docker volume.

2) backend – контейнер с серверной частью на Python 3.12 и фреймворке FastAPI. Предоставляет REST API для всех операций: CRUD-операции над персонами, поиск, статистика, импорт/экспорт. Взаимодействует с Neo4j через официальный Python-драйвер.

3) frontend – контейнер с клиентской частью на React 19 и Vite 8. Реализует одностраничное приложение (SPA) с маршрутизацией через react-router-dom. Граф родословного дерева отрисовывается с помощью Canvas API.

4.2. Используемые технологии

Компонент	Технологии
Backend	Python 3.12, FastAPI 0.111, Uvicorn 0.29, neo4j 5.26 (драйвер), Pydantic 2.7, pydantic-settings 2.2
Frontend	React 19, Vite 8, react-router-dom 7, Canvas API (визуализация графа)
База данных	Neo4j 5.18 Community Edition
Инфраструктура	Docker, Docker Compose
CI/CD	GitHub Actions (7 workflow-проверок)

4.3. Снимки экрана приложения

Описание экранов приложения со снимками представлено в Приложении Б (Инструкция для пользователя).

5. Выводы

Реализовано веб-приложение для фиксации и интерактивного картографирования наблюдений за животными. Обеспечены ключевые пользовательские сценарии, включая публикацию медиаконтента, геолокацию, социальное взаимодействие (лайки, комментарии) и управление базой данных. Система поддерживает функции поиска, фильтрации и интерактивной визуализации объектов на карте. В рамках дальнейшего развития платформы запланировано расширение аналитических модулей, оптимизация работы с картами высокой плотности и улучшение системы обработки мультимедиа.

Литература

1. Neo4j Documentation [Электронный ресурс]. – Режим доступа: <https://neo4j.com/docs/>
2. FastAPI Documentation [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com/>
3. React Documentation [Электронный ресурс]. – Режим доступа: <https://react.dev/>
4. Репозиторий проекта [Электронный ресурс]. – Режим доступа: <https://github.com/moevm/nsql1h26-tree>
5. Docker Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/>
6. Vite Documentation [Электронный ресурс]. – Режим доступа: <https://vite.dev/>
7. Cypher Query Language [Электронный ресурс]. – Режим доступа: <https://neo4j.com/docs/cypher-manual/current/>

Приложение А. Документация по сборке и развертыванию

Предварительные требования

Для развертывания приложения необходимо наличие установленных Docker и Docker Compose.

Порядок развертывания

1. Собрать и запустить контейнеры:

```
docker compose build --no-cache && docker compose up
```

2. Открыть Swagger API: <http://localhost:8000/api/v1/docs>

3. Открыть фронтенд: <http://localhost:5173>

4. Регистрация/вход:

Через фронтенд: зарегистрируйтесь (например, email: string@mail.ru, пароль: 123).

Или через Swagger: POST /auth/register, затем используйте куки/токен для авторизации.

5. Тестовые аккаунты:

```
vlad2005 / 123
```

```
kirill2005 / 123
```

```
alex2005 / 123
```

6. Примечания:

Если контейнеры не поднимаются — проверьте, что локальные папки не смонтированы в контейнеры (запрещено).

Приложение Б. Инструкция для пользователя

<https://github.com/moevm/nsql1h26-anim/wiki>

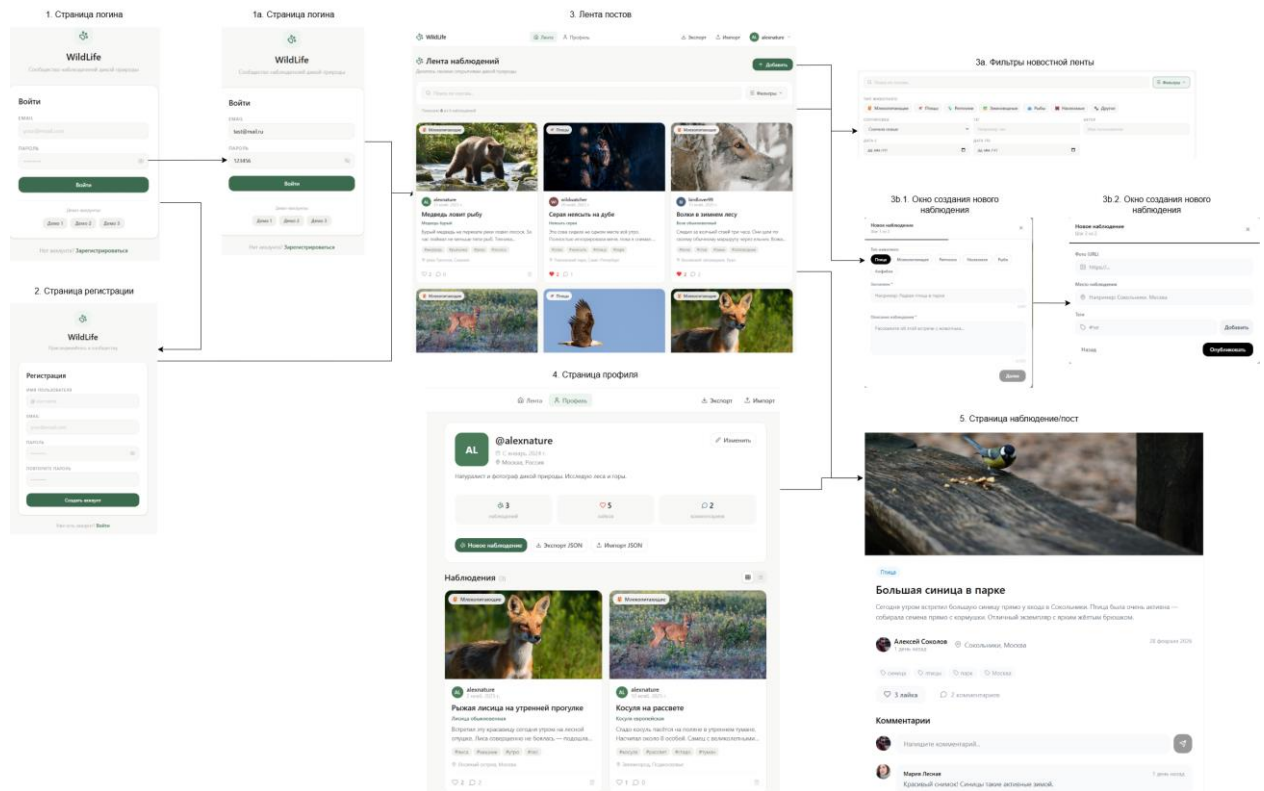


Рис. 3. Макет приложения

